MP2: User Interaction and Simple MVC

Due Time: Refer to the course web-site

Objective

In this programming assignment, we want to familiarize with implementing solutions with GUI, and designing small scale interactive solutions based on what we can implement for a MVC architecture in the Unity environment.

Approach

We will build a simple hierarchical editor. One where we can select, manipulate, and add new primitive GameObjects. Please refer to my attempt at the solution (WebGL build, simply click and run). Initially, your world will consist of a static plane (for friendly reference), **GrandParent**, **Parent**, and **Child** objects organized as follows.

- GrandParent:
  - Cube
  - blue-ish color
- Parent:
  - Sphere
  - Green-ish color
  - A child of GrandParent
- Child:
  - Cylinder
  - Red-ish color
  - A child of Parent

We will learn about this in the coming weeks, for now, just know that, GameObject.transform.localPosition: is the value in the Transform component, and GameObject.transform.Position is its position with the consideration of its parents.

- For example, if in the editor:
  - Parent's position in the Transform is: (x, y, z), and
  - Child's position in the Transform is (1, 1, 1),
- Then, in the script,
  - Child.transform.Position is (x+1, y+1, z+1)
  - Child.transform.localPosition is (1, 1, 1)

In your application at any point, you can click the left mouse button (LMB) to select any of the GameObject defined in the scene. LMB clicking on the static plane or an empty space results in selecting nothing. **Selected** object must:

- Be displayed in a unique color (mine is some kind of yellow-ish)
- Unselecting an object must cause it to return to its original color.
- Selecting any object will result in the displaying of an axis frame centered at the selected object. An axis frame is simply three cylinders, x-red, y-green, and z-blue axes showing the major axes of the Cartesian coordinate system. This axis frame is a convenient reference for the rotation axes.

You will create two simple GUI widget to support editing operations.

First GUI widget, **XformControl**: this widget allows the editing of the Transform component of the currently selected GameObject:

- **Name:** name of the GameObject currently under editing.
- **Mode:** mode of editing, either Translation, Scaling, or Rotation
- **Sliders**: x/y/z slider for controlling the current mode
    - You must use the same sliders to support all three transformation modes
    - Sliders must show actual numeric value in text
    - Slider ranges must change according to the current selected transformation mode:
        - translation between -10 to +10,
        - rotation between -180 to 180, and
        - scaling between 0.1 to 5.

At any point, you can create a new GameObject primitive *as the child* of the current selected GameObject with the **CreatePrimitive** widget. The **CreatePrimitive** widget should support three toggle-like buttons where clicking on each will create one of these primitives: Sphere, Cylinder and Cube. When creating a new object, let **mSelected** be the currently selected GameObject:

- If mSelected is null
    - Newly created GameObject will be in black
    - Located at (0.5, 0.5, 0.5), this means subsequently created objects will overlap, that is ok.
    - A new root object
- Else:
    - Mewly created GameObject will have **mSelected** as parent
    - If **mSelected** has other children
        - newly created will have the same materials as the siblings sharing the same **mSelected** parent
        - Newly created will be located at (0.5, 0.5, 0.5) offset from the last child in **mSelected**
    - Else
        - Newly created will be in cyan.
        - Newly created will be located at (0.5, 0.5, 0.5) offset from **mSelected**
    - Note: the above means, new root object will always be in black, and new leaf objects will always be in cyan.

In all cases, the new object must have a unique name (I used a global integer as part of the object name). There you have it, with the above, you have a functional but quite un-usable editor that is really good for not much. You do not need to support object deletion (requires policy decision, but should be rather straightforward to implement).

Hints:

1. My creation plane is 14x14x14 (Quad primitive) located at the origin. My Camera is located at (0. 7.5, -16), with rotation (22.5, 0, 0), and FOV of 39.6.

2. Take a look at: EventSystem.current.IsPointerOverGameObject(), when figuring out how to support selecting null when click on nothing. I googled: "*unity mouse click passing through UI*" and found this: https://forum.unity.com/threads/preventing-ugui-mouse-click-from-passing-through-gui-controls.272114/

3. Take a look at **Rendering Mode** of **Materials**, these must be set to ***Transparent*** to get transparency working. A color's alpha, or a, channel (r, g, b, a) controls transparency. Values of less than 1.0 is transparent. I use 0.5 for mine, my selected color is: (0.8, 0.8, 0.1, 0.5).

4. **Trouble with rotation:** Note that for position and scale, you are setting the values in the Transform component to that in the GUI slider. This is rather straightforward. However, this does not work for rotation. For rotation, you must *concatenate* subsequent rotations. Let *R(Theta, Axis)* be a rotation of Theta-degree along the Axis. Now, if select an object and perform R(10, X) [rotation of 10-degrees along the X-axis], and then, you select the object again to perform R(10, Y), what you

have done is:
- R(10, X) * R(10, Y)
- Note, this is different from setting the rotation angles to (10, 10, 0) (try it!)

Fortunately, we have Quaternion to save us. We will learn more about this, but for now,

- Each time an object is selected, you can display zero for the current rotation values.
- Compute slider movement for each session theta: receive a degree
- Do this:
  q = Quaternion.AngleAxis(theta, Axis)
  object.transform.localRotation = object.transform.localRotation * q
- DO NOT rotate with **EulerAngles:** you can try it, you will result in Gimbal lock and not know what is going on.

Some interesting (or not?) to note: if you edit the rotation an object by its EulerAngles, a few adjustments in x/y/z is all you have to make before the object will get into an orientation where you lost complete track of how to further adjust it. I will explain this better in class, but, the term we are looking for is: **Gimbal Lock**. We will learn much more about this in the next few lectures.

5. My "Radio buttons" for choosing between TSR is similar to that from MP1, implemented based on the Unity "ToggleGroup," (https://docs.unity3d.com/Manual/script-ToggleGroup.html). I found this by googling, "*how to radio button in unity*".

6. Give serious thoughts on how you want to separate responsibilities, define classes to hide implementation accordingly. We have a pretty clear Model (M), and pretty clear Controllers (C), no real View (V) yet.

7. Potential Unity bug: on my PC running Unity 2021.3.10f1, after creating two children for a node, sometimes selecting the first child will result in both of the children being turned yellow. It is tricky to reproduce this bug, but from what I can figure, Unity seems to have a bug with their material instancing caching, that sometimes, the two GameObject will continue to share the same material when they should not.

Credit Distribution:

1. **(5%)** Initial scene with the static plane, GrandParent, Parent, and Child
   - (5%) Reasonable size
   - (5%) Proper setup
   - **(-50%) If the initial setup is not entirely and conveniently visible at 1920x1080**

2. **(20%)** LMB Selection
   - (10%) Able to select nothing (clicking on the plane or nothing)
   - (5%) Proper color changes of the selected
   - (10%) Display of Axis Frame on the selected object
   - (10%) Able to restore color after de-selecting

3. **(40%)** XformControl
   - (5%) Proper and neat layout
   - (10%) Show current selected name (or null)
   - (10%) Proper support of the three modes: T, S, R
   - (10%) Proper adjustments by the three slider bars
   - (15%) Proper update binding:
     - When changing slider modes, newly selected mode is properly set to the values in the Transform. When object selection change, newly selected does not jump to previous GUI setting.
     - For rotation: newly selected object will show 0.0 on all slider bars, this is ok.

4. **(25%)** Created Primitive Behavior:

- (5%) Able to create three primitive types
- (5%) With unique name
- (10%) Proper parenting
- (10%) Proper color scheme
- (5%) Proper initial placement after creation

5. **(10%)** Submission and Others:
    - (5%) Submission include all source code and exe folder zipped
    - (5%) Submission with proper zipfile naming: FirstLastMP2.zip
    - (10%) Proper build to Windows EXE running in Resizable Windowed mode

Submission:

- **Source code**: just the one folder: **Assets**
- **Executable**: Include a separate **EXE** folder, and build your executable into this folder.
    - **Build** ➔Player Settings
        - **Fullscreen Mode**: Windowed
            - **Resolution: 1920x1080** is fine
        - **Resizable Window:** check this!

Extra Credit:

Again, there tons of stuff you can do. DO NOT over-do (e.g., by attempting to do everything that is in the Unity Editor). Some ideas:

- Support deletion: this is trivial
- Show the current hierarchy in some way: less easy
    - Allow selection of objects from the hierarchy shown: not too difficult
- Allow re-parenting: more challenging